

pyLensLib: A Python package for multi-scale gravitational lensing simulations

M. Meneghetti¹ and The pyLensLib Collaboration¹

INAF – Osservatorio di Astrofisica e Scienza dello Spazio di Bologna, Via Piero Gobetti 93/3, 40129 Bologna, Italy

Received XX XX 20XX / Accepted XX XX 20XX

ABSTRACT

Context. Strong and weak gravitational lensing analyses increasingly require flexible software that can combine analytical lens models, map-based mass reconstructions, source population synthesis, and realistic instrument effects in a single reproducible pipeline.

Aims. We present **pyLensLib**, an open-source Python package designed to simulate gravitational lensing across multiple physical scales, from stellar microlensing to galaxy and cluster strong-lensing regimes, with explicit support for galaxy-galaxy strong lensing (GGSL) applications.

Methods. The package is organized around two abstract foundations: lens models inherit from **genlen**, and source models inherit from **gensrc**. This design provides a shared interface for computing lensing fields, critical lines, caustics, and ray-traced images. **pyLensLib** combines analytical models (SIE, PIEMD, NFW-like and multipole/shear perturbations), map-based deflectors, multi-plane propagation, source/light models (Sersic, point sources, shapelets, spiral structures), and observation modules for PSF convolution and noise realization.

Results. We describe the package architecture, principal algorithms, module-level functionality, and practical simulation workflows. We also document optional interfaces to external simulation and modeling ecosystems (e.g., Gadget/Subfind readers, Lenstool interoperability, optional **lenstronomy** image-finding backend), together with performance-oriented components such as vectorized solvers and Numba-accelerated source rendering.

Conclusions. By unifying physically motivated lens modeling, synthetic imaging, and analysis utilities, **pyLensLib** provides a practical environment for reproducible lensing experiments and method development in current and upcoming survey contexts.

Key words. gravitational lensing: strong – gravitational lensing: micro – methods: numerical – methods: data analysis – galaxies: clusters: general

Use \titlerunning to supply a shorter title and/or \authorrunning to supply a shorter list of authors.

1. Introduction

Gravitational lensing has become a standard tool for studying mass distributions across a large dynamical range, from stellar compact objects to galaxy clusters. Modern lensing analyses frequently combine distinct tasks: (i) parametric lens modeling, (ii) map-based ray tracing from numerical simulations, (iii) realistic source population construction, and (iv) synthetic observation generation for direct data-model comparison. Maintaining these tasks inside a single software environment is valuable for reproducibility, consistency, and rapid methodological iteration.

pyLensLib is a scientific Python package developed to address this need. The codebase targets lensing at multiple scales, including microlensing, galaxy-scale lensing, and cluster-scale strong lensing with explicit support for GGSL-oriented workflows. The package is available under the MIT license and is structured to support both interactive use and scripted batch production.

This paper documents the current functionality of **pyLensLib** in a software paper format suitable for journals such as *Astronomy and Computing* or *Astronomy & Astrophysics*. The emphasis is on architecture, implemented capabilities, and end-to-end simulation logic.

2. Scientific and software scope

2.1. Design goals

The package design follows four operational goals:

1. **Multi-scale lensing coverage:** support for microlensing, galaxy lenses, and cluster lenses in one library.
2. **Composable physical models:** analytical and map-based deflectors can be combined and reused.
3. **Realistic mock imaging:** synthetic observations include PSF effects, background, and shot-noise realizations.
4. **Workflow interoperability:** interfaces to external data/model ecosystems are available when needed.

2.2. Problem classes addressed

The implemented functionality targets:

- strong-lensing image formation and critical-curve/caustic analysis,
- source-plane and image-plane morphology studies,
- lens-substructure perturbation experiments,
- map-based lensing from simulation snapshots,
- generation of mock data products for pipeline testing.

3. Core architecture

3.1. Base class hierarchy

`pyLensLib` is built around two abstract operational layers:

- `genlen` (`pyLensLib/genlen.py`) for lens-side quantities,
- `gensrc` (`pyLensLib/gensrc.py`) for source-side ray tracing.

Lens classes inheriting from `genlen` expose a shared interface for convergence, shear, deflection angles, and (optionally) potential-related quantities. Source classes inheriting from `gensrc` share ray-tracing logic against any compatible deflector. The classes `CriticalLine` and `Caustic` (`pyLensLib/critcau.py`) provide geometric containers for topological lensing structures.

3.2. Lensing equations and implemented fields

The package is based on the standard thin-lens mapping:

$$\beta = \theta - \alpha(\theta), \quad (1)$$

with Jacobian

$$\mathbf{A} = \begin{pmatrix} 1 - \kappa - \gamma_1 & -\gamma_2 \\ -\gamma_2 & 1 - \kappa + \gamma_1 \end{pmatrix}, \quad (2)$$

and critical lines defined by $\det \mathbf{A} = 0$. In practice, `genlen` computes κ , (γ_1, γ_2) , and α on user-defined grids, then extracts tangential and radial critical contours and maps them to caustics.

3.3. Documentation-level module grouping

To keep the API interpretable, the documentation is organized in eight groups (Table ??).

4. Lens mass modeling functionality

4.1. Analytical deflectors

The package includes a set of parametric lens models:

- `sie`: singular isothermal ellipsoid,
- `piemd`: pseudo-isothermal elliptical mass distribution,
- `nfwell`: pseudo-elliptical NFW-like halo implementation,
- `extshear`, `multipoles`, and `perturber` for perturbative components.

These models provide coherent access to convergence, shear, deflection angles, and, where applicable, potential-based derivatives.

4.2. Map-based deflectors

The `deflector` class ingests precomputed deflection maps (e.g., FITS products), enabling direct use of external mass reconstructions or numerical ray-tracing outputs. The resulting object remains compatible with the same source-model APIs used for analytical lenses.

4.3. Composite lenses and perturbative experiments

`compositeModel` supports additive composition of multiple lens components, allowing direct construction of “smooth + substructure” or “cluster + galaxy members” models in a common frame. This is central for GGSL and lens-perturbation studies.

4.4. Microlensing

Microlensing functionality is available in `microlensing`, with utilities for point-lens and binary-lens configurations and associated light-curve generation.

5. Source and light-model functionality

5.1. Sersic-based sources

Extended source modeling is provided by:

- `sersic`: general-purpose Sersic source renderer,
- `sersic_numba`: Numba-accelerated implementation for high-throughput runs,
- `sersic2`: alternative implementation path.

Core parameters include Sersic index, effective radius, axis ratio, orientation, redshift, and total flux normalization.

5.2. Point and basis-function sources

`pointsrc` provides point-source image-finding utilities and magnification-aware image construction. Basis-function and structured-light alternatives are available via `shapelets`, `shapelets_polar`, and `spiral`. `sourceplane` and `poststamp` support population-level and postage-stamp workflows.

6. Ray tracing and multi-plane propagation

6.1. Single-plane propagation

For standard thin-lens experiments, source objects call `gensrc.ray_trace()` against any compatible lens object, yielding lensed image-plane brightness fields and contour products.

6.2. Grid-based tracing from mass maps

`raytracer` provides map-based tracing utilities from projected mass distributions. `raymesh` complements this with adaptive mesh sampling utilities for computationally targeted ray placement.

6.3. Multi-plane lensing

`multiplane` implements sequential propagation through ordered foreground planes up to a source plane at z_s , enabling line-of-sight lensing experiments beyond the single-plane approximation.

7. Observation and instrument modeling

`observation` encapsulates forward modeling of observational effects:

Table 1. Main `pyLensLib` documentation groups and representative modules.

Group	Representative modules
Core foundations	<code>genlen</code> , <code>gensrc</code> , <code>critcau</code>
Lens mass models	<code>deflector</code> , <code>piemd</code> , <code>sie</code> , <code>nfwell</code> , <code>extshear</code> , <code>multipoles</code> , <code>perturber</code> , <code>compositeModel</code> , <code>rs</code>
Source and light models	<code>seraic</code> , <code>seraic_numba</code> , <code>seraic2</code> , <code>pointsrc</code> , <code>shapelets</code> , <code>shapelets_polar</code> , <code>spiral</code> , <code>so</code>
Ray tracing and multi-plane	<code>raytracer</code> , <code>raymesh</code> , <code>multiplane</code>
Simulation utilities	<code>cluster</code> , <code>gadget</code> , <code>perlin</code> , <code>perlin_parallel</code> , <code>samplers</code> , <code>maputils</code> , <code>subfind</code>
Catalogs/populations/external data	<code>srccatalog</code> , <code>photocatalogs</code> , <code>sedmodel</code> , <code>subhaloPop</code> , <code>lenstool</code>
Observation and instrument effects	<code>observation</code>
Interactive tools	<code>lens_interact</code> , <code>lens_interact_qt</code>

- magnitude-counts conversion,
- background level handling,
- PSF convolution methods,
- Poisson and Gaussian noise realizations.

This allows direct generation of synthetic imaging products from model images for pipeline tests and method benchmarking.

8. Simulation utilities and data interfaces

8.1. Simulation-side infrastructure

`cluster`, `gadget`, and `subfind` provide utilities to ingest and manipulate simulation-derived particle/halo data. `maputils` offers analysis tools for map-level operations (profiles, contour geometry, image moments). `samplers` and `Perlin` modules support structured synthetic sampling and texture/noise generation in forward simulations.

8.2. Catalog and SED-level tools

`srccatalog`, `photocatalogs`, and `sedmodel` support source population and spectral-energy-distribution workflows. `subhaloPop` provides subhalo-population utilities for controlled perturbation studies.

8.3. Interoperability

`lenstool` utilities support interoperability with `Lenstool`-style parameter and map products. Optional dependencies are used only where required (e.g., `Gadget/Subfind` readers, Qt tools, optional `lenstronomy` point-image backend), allowing lighter installations for core use.

9. Validation workflows and example use cases

The repository includes extensive scripted examples under `Test/`, covering model sanity checks, ray tracing, deflector handling, observation simulation, and source catalog generation. A representative functionality test is the giant-arc workflow:

1. build a main deflector at fixed (z_l, z_s) ,
2. generate aligned extended source emission,
3. produce lensed image and synthetic observation,
4. inject subhalos at selected masses and locations,
5. compare perturbed and unperturbed residual maps.

Such workflows are intended for method development and sensitivity studies in substructure lensing analyses.

10. Performance considerations

`pyLensLib` relies on vectorized NumPy/SciPy operations for most field computations and interpolation tasks. Numba acceleration is available for selected hot paths (notably `Sersic` source rendering). Additional acceleration opportunities exist for large-map campaigns, including cache-aware PSF handling, reduced reallocation patterns in map/particle workflows, and broader JIT coverage of iterative kernels.

11. Limitations and development priorities

Current functionality is broad but heterogeneous across modules because the package supports multiple scientific use cases and legacy workflows. Practical priorities for continued development include:

- expanded automated test coverage with strict CI gating,
- stronger typing and parameter validation across public APIs,
- further harmonization of naming conventions across legacy/newer modules,
- optional GPU-oriented pathways for large simulation campaigns.

12. Conclusions

`pyLensLib` provides an integrated environment for gravitational lensing simulations across microlensing, galaxy-scale, and cluster-scale regimes. Its main strength is a consistent architecture that links physically motivated lens/source models to realistic synthetic observations and analysis utilities. This structure makes the package suitable for both exploratory lensing studies and reproducible forward-modeling pipelines aligned with modern survey needs.

Acknowledgements. The authors acknowledge the open-source scientific Python ecosystem, including NumPy, SciPy, Astropy, Matplotlib, and related community projects that underpin `pyLensLib`.

Code availability

`pyLensLib` is distributed under the MIT License in the project repository: <https://github.com/meneghetti/pyLensLib>

References

Astropy Collaboration, Robitaille, T. P., Tollerud, E. J., et al. 2013, *A&A*, 558, A33

- Astropy Collaboration, Price-Whelan, A. M., Sipocz, B. M., et al. 2018, *AJ*, 156, 123
- Bartelmann, M., & Schneider, P. 2001, *Phys. Rep.*, 340, 291
- Einstein, A. 1936, *Science*, 84, 506
- Foreman-Mackey, D., Hogg, D. W., Lang, D., & Goodman, J. 2013, *PASP*, 125, 306
- Harris, C. R., Millman, K. J., van der Walt, S. J., et al. 2020, *Nature*, 585, 357
- Lam, S. K., Pitrou, A., & Seibert, S. 2015, in *Proc. Second Workshop on the LLVM Compiler Infrastructure in HPC*, 1
- Navarro, J. F., Frenk, C. S., & White, S. D. M. 1996, *ApJ*, 462, 563
- Navarro, J. F., Frenk, C. S., & White, S. D. M. 1997, *ApJ*, 490, 493
- Meneghetti, M., et al. 2021, *Science*, 369, 1347
- Sersic, J. L. 1968, *Atlas de Galaxias Australes* (Cordoba: Obs. Astron., Univ. Nac. Cordoba)
- Virtanen, P., Gommers, R., Oliphant, T. E., et al. 2020, *Nat. Methods*, 17, 261